Arthur: Repository Robustness Review

Nov 4, 2026 · @Connor Beaton

Summary

Three things crash today, several more will crash on the first odd OANDA response, and the rest is structure and hygiene. Nothing in the repo was changed; this is a read-only review of `cbeat0n/ArthurLVL2`.

Update to my earlier notes. The Oct 1 Claude Code rewrite fixed the zero-cost backtest, the row-alignment risk and the causality question. `causal_signal` replays the live window per bar and I verified it is causal.

Still open from before:

- `decisionrule()` returns constant HOLD (`if True: return 2`). -> Will be fixed in trading section, not in this document
- `detect_minor_anomaly` is a stub that returns False. -> Modified RiskManager class with multiple helpers instead of standalone function to find error / unknown codes or high pressure z scores. Modified *market*_cycle and futures to independently call errors. Modified Arthur.py to assist as well.
- `in_bhe_hours` has `devtime = True`. -> Method replaced with two methods that finds UTC Chicago hours converted from NY hours
- The account ID is hardcoded in `Arthur.__init__`. -> Leaving it hardcoded.
- `cancel_order` is a no-op. -> Writing this with trading logic, not in this document.

Findings are marked **confirmed** (reproduced by running a synthetic check) or **read** (found by reading the code).

Crashes today

Three defects fail on ordinary input. All three were confirmed by running them.

1. **Invalid horizon `M1` and `M10`.** `Arthur.py:76` defaults the horizon to `"M1"`, and `templates/homepage.html:77` offers `<option value="M10">`. Neither matches the `MXQN` pattern in `utils/granularity.py`, so the parser rejects both. Fix: default to `M1Q0`, and make `M10` an `M10Q0` option.

Both of these were oneliners by fixing the strings to be syntaxed with "M1Q0" or "M10Q0"

2. **Empty candle response.** When OANDA returns no candles (weekend, closed market), the tabular step raises `KeyError`. `safe_timestamp` in `utils/transform_cleaned.py` also uses `.iloc[-1]`, which fails on empty input. Fix: return early with an empty frame and skip the cycle.

   transform_cleaned.py now checks for empty and passes along an empty df. _*acquire*_market and market_cycles now use guardrails for empty strings for files and a lack of candles.
3. **Candle count is 6x off at S30.** `_acquire_market` in `utils/operators.py:230` computes `int(8 * zone * horizon_minutes * 60 * 0.2)`, which assumes S5 bars. At S30 it requests six times too many bars. Line 241 also calls `clean_tabular(tabular_data, self.config.horizon)` without passing the intake, so it resamples as S5. Fix: derive the count from `INTAKE_SECONDS` in `utils/granularity.py` and pass the intake through. Something in the refresh was still non functional after first generation fix,

   In operators.py, granularity.py was imported and the standard ReGex parse was exchanged for granularity calculations. Minor bugfixes on the UI were implemented to push this change forward. Bug no longer present as evidenced by M1Q0 and S30.

Latent crashes

These work until OANDA returns something unexpected. All five were found by reading.

4. **Unchecked HTTP responses.** `fetch_candles` in `utils/extract_quantitative.py` has no timeout and does not filter `complete`, so a half-formed candle enters the signal. `fetch_historical_candles` in the backtester filters `complete` but also has no timeout. Fix: one shared session helper with a timeout, retries and a `complete` filter.

   Fixed the unchecked HTTP responses via guardrails with a helper module named http_helpers.py for a class of Sessions and json/raise methods that add retries and stop complete shutdowns.

5. **Functions that return None.** `Trader.close_trade` (`utils/operators.py:439-444`) and `make_order` (`475-481`) return None on an unexpected response, and `_ignition_A` (`168-177`) can too. Callers then index into None. Fix: raise a typed error or return a result object.

   Functions that return None were changed to have resultant objects be returned instead. Guardrails were put in so that they return a string such as 'unknown' and Trader and Arthur class were modified in operators.py.
6. **SELL path assumptions.** `decision_loop` (`utils/operators.py:512-537`) uses `.index()` on the positions list, which raises `ValueError` when the instrument is absent. It also sends a float lot (`0.925 * currentLong / N`) and reads `order["status"]` directly. Fix: look the position up by name, round to integer units, and use `.get`.

   Fixed by adding a new method, *long*_units that looks at how many units long we are and modified _execute to use integer values.
7. **Cycle timing.** In `_market_cycle`, `cycletime` is computed at line 297 before `as_completed`, so it excludes the work it is meant to time.

   Due to concurrent and simultaneous bugging, see number 12.
8. **`finalize_records`.** `utils/transform_auto.py` has six `status_code == 200 else raise` blocks per branch, so one transient failure aborts the whole finalization. Fix: retry, then log and skip that record.

   Used a patch to merge the branches and bugfix for errors showing up. status_code == 200 else raise blocks fixed.

Structure

These do not crash, but they make the code hard to trust or change. All were found by reading except item 9, which `diff` confirmed.

9. **Duplicated `finalize_records`.** `utils/transform_auto.py` holds two identical branches (lines 57-229 and 237-409). Collapse them into one function with the branch as a parameter.

Same as 8, patch used to fix branching for changing into a single branch.

10. **Backtester import in the live path.** Line 14 of `utils/transform_auto.py` imports `pip_size` from `weeklybacktesting.backtestingprocess`. Move it to a shared module, since live code should not depend on backtest code.

    See item 11. Overlap included in patch.

11. **`pip_size` special-cases only USD_JPY.** `weeklybacktesting/backtestingprocess.py:50-64` returns 0.01 for USD_JPY and 0.0001 for everything else. That is wrong for any other JPY cross and breaks multi-currency later.

    Added a patch that includes PIPS that are SUPPORTED that can be used (EUR_USD, USD_CHF , USD_MXN). Modified files included.

12. **Reload drives trading.** `vault_storage.html` and `market_view.html` call `location.reload()` every horizon, and that reload runs the trading cycle. A closed tab stops the bot. Move the loop into a server-side scheduler.

    Item 7: the browser reload drove trading

    - **Before:** trading cycles ran inside the `/accounting` request, so they only happened while a browser tab was open and reloading. A closed tab or a throttled background tab meant no trades.
    - **New `utils/scheduler.py`:** `CycleScheduler` is a background thread that fires on UTC-midnight-aligned multiples of the horizon, plus a 2 s offset. It never overlaps cycles and skips any boundary it missed instead of running late. Exceptions are caught and counted, and it stops when `run_cycle` returns False.
    - **`Arthur.run_once()`:** one trading cycle (acquire market data, *`ignition`* `A`, *`market`* `cycle` if OPEN) that returns the operational state.
    - **`scheduled_cycle()` in `Arthur.py`:** calls `run_once()` and stores the state. On CLOSED it shuts Arthur down, POSTs `/shutdownserver` and tells the scheduler to stop.
    - **`/accounting`:**
        - On the first visit it builds `Arthur` and starts the scheduler, under `START_LOCK`, so simultaneous first visits start only one.
        - It checks the intake/horizon pairing and returns 400 if it's invalid.
        - After that it only handles the state switch and shows account info.

- The vault page shows the scheduler status: last and next cycle (UTC), cycles run, failed in a row, and last error.

Item 12: `cycletime` was computed too early

- **Before:** `cycletime` was measured before all the manager futures had finished, so it understated the real cycle length.
- **Now:** it's computed after every future completes, so it equals the slowest trader.
- **Related hardening:** each `f.result()` is wrapped in try/except. One failing trader becomes an `error` status and doesn't lose the others' results.

13. **`/accounting` both renders and runs a cycle** (`Arthur.py`, `_market_cycle`). A GET that places orders is unsafe to refresh or prefetch.

    Added a patch that changes the GET to a POST for the action accounting. This runs on a /start function with Flask now. It is read only. Includes tickbox so live users must check it to understand they are using real money.
14. **Account setup.** `Arthur.__init__` hardcodes `"101-001-37152677-001"` (lines 134-135), and `k = num_traders // num_managers` (line 120) is zero when traders are fewer than managers.

Guards before real money

None of these matter on the demo account. All matter on a live one. All were found by reading.

15. **`/shutdownserver` is an unauthenticated POST** (`Arthur.py`). Anyone who can reach the port can stop the bot.

    Replaced /shutdownserver with stop_server() which is called elsewhere.
16. **`/visualize/` reads any path.** It calls `pd.read_csv(request.args.get("signalFile"))` at `Arthur.py:163-166`. Restrict it to files under `data/`.

    Fixed with an earlier patch from fixing market_view.html.
17. **Live mode comes from a query string.** `mode = request.args.get("mode", "Demo")` at `Arthur.py:80` lets a URL switch to Live. Take the mode from config, and require an explicit confirmation for Live.

Added a patch that creates a switch called ALLOW_LIVE which is next to STOP_WHEN_CLOSED. templates/homepage.html opens only when ALLOW_LIVE is True.

18. **No lock on `GLOBAL_STATE`.** A Flask thread and the `ThreadPoolExecutor` share one dict. Add a lock, and make order placement idempotent so a repeated cycle cannot double-buy.

    Patched in updates that stop GLOBAL_STATE from being written over simultaneously by other parts of Arthur being ran under the hood.

19. **Safety stubs.** `RiskManager.in_bhe_hours` has `devtime = True`, `detect_minor_anomaly` always returns False, `gate_lotsize` is static, and `cancel_order` returns None. Each is a switch that looks on and is off.

    Added safety_stub patch that raises an error for cancel_order , changed devtime , and added vault page cards for each manager

20. **Reduce-only sells.** `OandaTrading.make_order` sends `positionFill: REDUCE_ONLY` for sells. The fill reply may report `tradeOpened` instead of `tradeReduced`, which would be mis-read as a close. This needs one demo fill to confirm, so I have not asserted it.

    Fixed with Trader.make_order reporting units, and cancelling orders that are too small. Execute function also closes trade after buy as temporary guardrail.

21. The dev server (`ArthurFlask.run(debug=False)`) is not a production server, and none of the routes above require authentication. The Waitress line is commented out and its comment shows `0.0.0.0`, so choose the bind address deliberately when you switch.

    Added a switch for debug with Werkzeug and Waitress. Gunicorn and Docker coming much later.

22. Before signal and then trading, go through the front end and polish all 3 .html files. -No more reload-and-resubmit: the vault page now refreshes its summary in the background, and the switch form redirects after you press Apply. Reloading can't resend the switch or show a browser error page, and the dropdown shows the current state.

-Safer refreshing: requests time out, don't stack, and back off while the server is failing. They catch up when you return to the tab, and the vault page never polls OANDA faster than every 15 s. Closed asks for confirmation, and Apply and Start can't be double-clicked.

-Caching: the Arthur.py hook sends no-store on pages and data, and static CSS is re-checked on every load, so the back button can't show an old account and CSS edits show up immediately.

-Clearer status: market view turns the status line red when the newest bar is older than three horizons, and shows a message if Plotly fails to load. The vault gets a "Manual switch" card, and the homepage disables invalid horizon options and explains when Live is off.

-Housekeeping: added viewport tags, DOCTYPE and label fixes, the security headers (X-Frame-Options, nosniff, Referrer-Policy), and dropped the unused Plotly script from the vault page. A follow-up patch fixed the collapsed charts caused by responsive: true.

Repo hygiene

23. **`data/` is not in `.gitignore`.** The repo is public, so runtime CSVs (candles, signals, trade logs) get committed. Add `data/` to `.gitignore` and remove it from tracking with `git rm -r --cached data`. `.env` is already ignored.

    Fixed with minor patch that keeps data on the harddrive instead of pushing it to repository.
24. **Non-atomic writes.** The `utils/load_*.py` files call `to_csv` directly, so a crash mid-write leaves a truncated file that `read_csv` then fails on. Write to a temp file and `os.replace` it.

    Added a small patch that helps with a helper module that makes atomic rewrites instead of to_csv to not have the system get bloated while running.

The repo also has no tests. A few `pytest` cases would cover the findings above: horizon parsing, empty candles, candle count per intake, and `causal_signal` on short and flat windows.

Checked and fine

- **Signal on bad windows.** `transform_to_signal` does not crash on short or flat windows (confirmed by running); NaNs are guarded and the final `fillna` covers the rest.
- **Causality.** `causal_signal` replays the live window (8 x zone horizon bars) per bar and uses no future rows (confirmed by running).
- **`utils/granularity.py`.** One clean source of truth for intake and horizon parsing; the fixes above should reuse it.
- **`generic.py`.** The re-export shim does its job of avoiding circular imports.
- **Modulo horizon.** The `60X + 15N` decomposition is intentional, not a bug.
- **Secrets.** `.env` is gitignored. I did not open the `.env` that was in `first.zip`.

Where to start

Fix the three crashes first, then the latent ones, then structure and hygiene. Each step is small enough to review alone.

1. **Items 1-3.** Horizon default and option, empty-candle guard, intake-aware candle count. Add the `pytest` cases for these in the same change. -> Done.
2. **Items 4-8, plus the lock and idempotency check in 18.** Shared request helper, no silent None returns, safe SELL lookup, cycle timing, retrying `finalize_records`.
3. **Items 9-13 and 21-22.** Collapse the duplicate branch, move `pip_size`, stop the reload from driving trades, and fix `.gitignore` and atomic writes.

Items 15-17 and 19-20 wait until you move toward a live account. The signal itself (`decisionrule()`) is the next job after robustness, per your README order.